



← Back to student guides

 ClearML

Experiment tracking

3 levels

116 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

TASK

QUEUE

AGENT

MODEL

17

sections

35

examples





is repeated for its own sake. where you already know the basics, we go straight to the depth.

Where this picks up

Topic you already use	What this level adds
<code>Task.init</code>	Task types, <code>Task.create</code> vs <code>init</code> , <code>continue_last_task</code> , reuse semantics, distributed runs
Automatic capture	Exactly what the framework patches hook, and how to fix detected requirements
<code>task.connect</code>	Sections, nested objects, <code>connect_configuration</code> round-trips, and <code>\${...}</code> references
Logger	Custom plots, tables, media, iteration offsets, and reporting from a subprocess
Artifacts	Streaming, <code>wait_on_upload</code> , storage backends, and deleting them safely
Models	The registry as a promotion workflow : tags, publish, lineage, and ClearML Serving
Datasets	Squashing, parent chains, cache strategy, and dataset-level metrics
Agents	Virtualenv vs docker mode, caching layers, <code>--services-mode</code> , and CPU/GPU pinning
Queues	Priority, multi-queue workers, the services queue, and autoscaling
Pipelines	Controller vs decorator in depth, caching, retries, monitoring, and parameter plumbing
Debugging	<code>clearml-session</code> , log levels, reproducing an agent environment locally



new

HPO with [HyperParameterOptimizer](#) · [clearml-task](#) · ClearML Serving ·
Reports · CI/CD

Each section starts with the problem it solves, and ends with a **Try it** you can do on a real project in a few minutes.

Task creation, reuse, and lifecycle

Beginner used one entry point: `Task.init`. At this level you need three, and knowing which is which removes a whole category of confusion.

PYTHON

Copy

```
# 1. init – "this process IS the task". Patches frameworks, captures the environment.
task = Task.init(project_name="vision", task_name="train")

# 2. create – "build a task object for someone ELSE to run". No patching, no capture.
child = Task.create(
    project_name="vision",
    task_name="train on the new split",
    script="src/train.py",
    repo="https://github.com/org/repo",
    branch="main",
    requirements_file="requirements.txt",
    docker="nvidia/cuda:12.1.0-runtime-ubuntu22.04",
)
Task.enqueue(child, queue_name="gpu")

# 3. clone – "copy an existing task as an editable draft"
clone = Task.clone(source_task="8f2c4b19...", name="lr=1e-3")
clone.set_parameter("General/lr", 1e-3)
Task.enqueue(clone, queue_name="gpu")
```



<code>Task.init</code>	Yes	Yes	Yes	Your own training script
<code>Task.create</code>	No	No. You declare it	No	Launching a script from a controller
<code>Task.clone</code>	No	Inherited	No	Reruns with different parameters

Two lifecycle behaviours surprise people, and both are documented but easy to miss:

Reuse. `Task.init` with default settings may `reuse` the last task created by the same script in the same project if that task has no recorded iterations or artifacts. It exists to stop a debugging session producing forty empty tasks. During a sweep it looks like your runs are vanishing, so pass `reuse_last_task_id=False`.

Continuation. `continue_last_task=True` (or `=task_id`) resumes an existing task, appending scalars from the last reported iteration instead of restarting at zero. That is the right tool for a job that got OOM-killed at epoch 40 of 100, and the wrong tool for a fresh experiment, because the scalar history will be a splice of two runs.

PYTHON

Copy

```
task = Task.init(  
    project_name="vision",  
    task_name="resnet18",  
    continue_last_task=True,          # resume, appending iterations  
    reuse_last_task_id=False,        # never silently reuse a different run  
)
```

Task types are more than decoration. They drive UI filtering and, for pipelines and services, actual behaviour:



training	The default for experiments
data_processing	ETL and dataset creation; keeps it out of the experiment table
testing / qc	Evaluation runs, so they are filterable separately
inference	Batch scoring jobs
controller	A pipeline controller. Long-lived, low-resource
service	Something that runs continuously on the services queue
optimizer	An HPO controller

A distributed run needs one task, not one per rank

Under `torchrun` or `accelerate`, every rank executes your script. If each calls `Task.init` you get eight tasks, eight sets of scalars, and a UI you cannot read. Initialise on rank zero and use `Task.current_task()` elsewhere, or let ClearML's own distributed support attach the workers to the master task.

distributed pattern

Copy

```
import os
from clearml import Task

rank = int(os.environ.get("RANK", 0))
task = Task.init(project_name="vision", task_name="ddp") if rank == 0 else None

# Later, anywhere in the codebase:
current = Task.current_task() # the task on rank 0, None elsewhere
if current:
    current.get_logger().report_scalar("loss", "train", loss, step)
```

► TRY IT

1. Run the same script twice in a row without `reuse_last_task_id=False` and check whether you got one task or two.



not modify at all.

4. Kill a run at iteration 50, then relaunch with `continue_last_task=True` and read the scalar chart.

Expected result – reuse only kicks in for empty tasks, which is exactly when you want it. Step three is the important one: `Task.create` is how you drive code that has no ClearML in it, and it is the foundation of both `clearml-task` and pipeline steps.

Automatic capture: the mechanism, and its edges

Beginner said "automagic". At this level you have to fix it when it goes wrong, so you need the mechanism.

On `Task.init`, the SDK installs **binding patches**. Each supported framework has a binding module that wraps specific functions:

Framework	Patched	Result
TensorBoard / TensorBoardX	<code>SummaryWriter.add_*</code>	Scalars, histograms, images, text
PyTorch	<code>torch.save</code>	Every checkpoint becomes an output model
Keras / TF	<code>Model.save</code> , callbacks	Models plus TB scalars
joblib / scikit-learn	<code>joblib.dump</code>	Output model
XGBoost / LightGBM	<code>save_model</code> / <code>Booster.save</code>	Output model
matplotlib	<code>pyplot.show</code> / <code>savefig</code>	A plot on the task



argparse	parse_args	hyperparameters under Args
hydra	The main decorator	The full resolved config
joblib / loky	Process start	Child processes attach to the same task

Requirements detection is separate and works by inspecting `sys.modules` **at the end of the run**, mapping imported modules to installed distributions. That yields three failure modes worth knowing:

01 A lazily imported package is missed

Imported inside a function that never ran, so it is not in `sys.modules`. The agent then fails with `ImportError`. Import at module level or declare it.

02 The wrong index is used

A detected `torch==2.3.0` installs the CPU build from PyPI, not the CUDA build from the PyTorch index. Docker mode with a CUDA base image is the real fix.

03 A local or private package is unresolvable

Detected as a name the agent cannot install. Vendor it into the repo, or make the private index available to the agent.

You can take control:

PYTHON

Copy

```
# Declare the environment explicitly rather than relying on detection
task.set_packages("requirements.txt")          # a file
task.add_requirements("torch", "2.3.0")        # a single pin
task.add_requirements("-r requirements-gpu.txt") # pass-through to pip

# Or force docker mode from the code
task.set_base_docker(
    docker_image="nvidia/cuda:12.1.0-runtime-ubuntu22.04",
```



BASH

Copy

```
# On the agent side, ignore detection entirely and trust the container
clearml-agent daemon --queue gpu --docker my-registry/train:2.3.0 \
  --force-current-version
```

clearml.conf on the agent: the settings that matter

Copy

```
agent {
  package_manager {
    type: pip
    system_site_packages: true      # trust what the container already has
    force_upgrade: false
    extra_index_url: ["https://download.pytorch.org/whl/cu121"]
  }
}
```

system_site_packages: true is the single most useful agent setting

In docker mode with a prepared image, it tells the agent to use the container's existing packages instead of reinstalling everything into a fresh virtualenv. Setup time drops from minutes to seconds, and you stop fighting CUDA wheel resolution, because the image already resolved it.

► TRY IT

1. Look at a completed task's Execution → Installed Packages and compare it against your `pip freeze`. It lists what you

Expected result – imported
, not everything installed.

2. Move one import inside a function, rerun, and confirm the package disappears from the list.
3. Call `task.set_packages("requirements.txt")` and confirm the list is now exactly your file.
4. Run an agent in docker mode with `system_site_packages: true` and time the setup phase against virtualenv mode.



that timing difference is the whole argument for prepared images at this level.

Configuration: sections, objects, and references

Beginner connected one flat dict. Real projects have layered config, and ClearML has three answers.

PYTHON

Copy

```
# Sections keep unrelated knobs apart in the UI
task.connect({"lr": 3e-4, "epochs": 20}, name="optimizer")
task.connect({"layers": 18, "width": 64}, name="architecture")
task.connect({"num_workers": 8, "prefetch": 2}, name="dataloader")
```

Each **name** becomes its own collapsible section, so a clone's Configuration tab is navigable instead of being a wall of forty keys.

For anything nested, **connect_configuration** is the right tool, and it round-trips both directions:

PYTHON

Copy

```
# Local run: reads the file, uploads its contents, returns the local path
# Agent run: downloads the server's (possibly edited) content to a temp file,
#             returns THAT path — so a UI edit changes behaviour with no code change
config_path = task.connect_configuration("configs/model.yaml", name="model")
cfg = yaml.safe_load(open(config_path))

# A dict works too, and comes back as a dict
cfg = task.connect_configuration({"arch": "resnet18", "head": {"dropout": 0.1}}, name="m
```



Shape	Flat key/value	Arbitrarily nested, or a whole file
UI	Editable rows	An editable text blob
Returns	The dict, remote-overridden	A path for a file, a dict for a dict
Use for	Hyperparameters you sweep	Model definitions, feature lists, prompts

Parameter **references** are what make pipelines and HPO able to rewire a task without editing code:

PYTHON

Copy

```
# In a pipeline controller
pipe.add_parameter("lr", 3e-4)
pipe.add_step(
    name="train",
    base_task_name="resnet18 baseline",
    parameter_override={
        "General/lr": "${pipeline.lr}", # from the pipeline's own params
        "General/data": "${prepare.artifacts.dataset.url}", # from a previous step
        "Args/epochs": "${pipeline.epochs}",
    },
)
```

Reference	Resolves to
<code>\${pipeline.NAME}</code>	A pipeline-level parameter
<code>\${STEP.parameters.General/x}</code>	Another step's parameter
<code>\${STEP.artifacts.NAME.url}</code>	Another step's artifact URL
<code>\${STEP.models.output.-1.url}</code>	That step's last output model
<code>\${STEP.id}</code>	The step's task id

Parameter names are section-qualified, and getting it wrong fails silently



not exist is **not** an error. It is added as a new, unread parameter, and your step runs with its defaults. Check the executed task's Configuration tab to confirm the override landed.

► TRY IT

1. Split one flat dict into three named sections and compare the Configuration tab before and after.
2. Connect a YAML file, then clone the task, edit the YAML

Expected result – in the UI

, and enqueue. Confirm the agent used your edited version.

3. In a two-step pipeline, override a parameter with `${pipeline.x}` and verify it landed by reading the step task's config.
4. Now misspell the target key on purpose and observe that nothing errors.

Expected result – step two is the one worth internalising: editing a config file in a browser and having a remote run pick it up is the feature that makes ClearML usable by people who do not touch the repository. Step four shows you the silent failure that pairs with it.

Agents in depth: modes, caching, and pinning

The agent is the piece with the most operational surface, and understanding its three phases explains every "why is setup slow" question.



1 Claim

Long-polls the API server for the highest-priority task on its queues.



2 Environment

Virtualenv mode: create a venv, install the package list. Docker mode: pull the image, then optionally reuse its site-packages.



4 Execute

Run the entry point, streaming stdout/stderr and machine metrics to the server.

5 Report

Upload artifacts and models, set the final status, and go back to polling.

Two modes, and the choice matters more than any other agent decision:

✓ Docker mode

- System libraries, CUDA, and drivers come from the image
- Reproducible: the image digest is recorded on the task
- Isolated between jobs, so a bad run cannot poison the host
- With `system_site_packages`, setup is seconds

✗ Virtualenv mode

- Only Python packages are managed; system libs are the host's
- CUDA wheel resolution is a recurring fight
- Faster to start on a pre-warmed host, and needs no Docker
- Fine for CPU jobs and for a laptop agent

The caches are what make the second run fast. There are four, and they are separate:

Cache	Holds	Config key
pip / package cache	Downloaded wheels	<code>agent.pip_download_cache</code>
venv cache	Whole prebuilt virtualenvs, keyed on the requirement set	<code>agent.venvs_cache</code>
git cache	Bare repository clones, so fetches are incremental	<code>agent.vcs_cache</code>
storage cache	Datasets, artifacts, and models fetched by the SDK	<code>sdk.storage.cache</code>



```
pip_download_cache { enabled: true, path: "/mnt/cache/pip" }
venvs_cache { max_entries: 20, free_space_threshold_gb: 20, path: "/mnt/cache/venvs" }
docker_apply_cuda_version_runtime: true
package_manager {
  type: pip
  system_site_packages: true
  extra_index_url: ["https://download.pytorch.org/whl/cu121"]
}
}
sdk { storage { cache { default_base_dir: "/mnt/cache/clearml" } } }
```

Running workers, in the four shapes you will need:

BASH

Copy

```
# One worker per GPU, both on the same queue
clearml-agent daemon --queue gpu --gpus 0 --detached
clearml-agent daemon --queue gpu --gpus 1 --detached

# Multi-GPU jobs: pin two GPUs to one worker
clearml-agent daemon --queue gpu-big --gpus 0,1 --detached

# Queue priority: drain "urgent" before "gpu"
clearml-agent daemon --queue urgent gpu --detached

# The services queue: many light, long-lived tasks on one CPU box
clearml-agent daemon --queue services --services-mode --cpu-only --detached
```

--services-mode is the one worth explaining. Normally an agent runs one task at a time. In services mode it launches each task as a background container and immediately goes back to polling, so a single small machine can host dozens of pipeline controllers, HPO optimisers, and monitoring tasks concurrently. Controllers spend their lives waiting, so they must never occupy a GPU worker.

Never run pipeline controllers on your GPU queue

A controller is a task that mostly sleeps while its steps run. Put it on a GPU worker and it holds that GPU for the pipeline's entire duration doing nothing, while its own GPU steps



► TRY IT

1. Run an agent in virtualenv mode and time the setup phase from the console log. Run the same task again and time it once more.
2. Enable `venvs_cache` and repeat. Compare all three numbers.
3. Start two agents on one queue and enqueue three tasks. Watch the third wait.
4. Start a services-mode agent and enqueue three pipeline controllers. Confirm all three run at once.

Expected result – the venv cache is usually the biggest single win: a repeated requirement set goes from a multi-minute install to a copy. Step four shows why `--services-mode` exists: without it, three controllers on one worker run strictly one after another.

clearml-task : launching code with no SDK in it

You will often need to run a repository you cannot or should not modify: someone else's training script, an open-source benchmark, a colleague's branch. **clearml-task** builds a task from the outside.

BASH

Copy

```
clearml-task \  
  --project vision \  
  --name "external repo baseline" \  
  --repo https://github.com/some/repo.git \  
  --branch main \  
  --script train.py \  
  --args lr=0.001 epochs=30 \  
  --requirements requirements.txt \  
  --docker nvidia/cuda:12.1.0-runtime-ubuntu22.04 \  
  --queue gpu
```



tracked, comparable, cloneable, and re-launchable. Automatic capture of TensorBoard scalars still works, because the agent injects the SDK before execution.

Flag	Does
<code>--repo / --branch / --commit</code>	What to clone
<code>--script</code>	The entry point, relative to the repo root
<code>--args</code>	Command-line arguments, recorded as editable parameters
<code>--requirements</code>	Explicit environment, instead of detection
<code>--docker</code>	Base image, and therefore CUDA and system libraries
<code>--queue</code>	Enqueue immediately; omit it to leave a draft
<code>--import-offline</code>	Import a previously recorded offline run

The related trick is **offline mode**, for a machine with no network access to the server:

PYTHON

Copy

```
from clearml import Task
Task.set_offline(offline_mode=True)
task = Task.init(project_name="vision", task_name="air-gapped run")
# ... trains normally, writing everything to a local zip
```

BASH

Copy

```
# Later, from a connected machine
clearml-task --import-offline /path/to/offline-<id>.zip
```

► TRY IT

1. Pick any public training repository and launch it with `clearml-task` against your local agent.



else's code from a browser.

4. Run something in offline mode, then import the zip and check that scalars survived.

Expected result – tracking and re-execution for code that knows nothing about ClearML. Step three is the point: the "clone, edit, enqueue" loop is not a property of your script, it is a property of the platform.

Hyperparameter optimisation

Beginner ran three learning rates by hand. `HyperParameterOptimizer` does that properly: it clones a base task N times, sets parameters from a search strategy, enqueues them, watches the objective, and kills the runs that are losing.

hpo.py

Copy

```
from clearml import Task
from clearml.automation import (
    HyperParameterOptimizer, DiscreteParameterRange,
    UniformParameterRange, UniformIntegerParameterRange,
)
from clearml.automation.optuna import OptimizerOptuna

task = Task.init(
    project_name="vision",
    task_name="resnet18 HP0",
    task_type=Task.TaskTypes.optimizer,
    reuse_last_task_id=False,
)

optimizer = HyperParameterOptimizer(
    base_task_id="8f2c4b19e0a7d3f1c6b8a2e4d7091f3b", # a completed, working run
    hyper_parameters=[
        UniformParameterRange("General/lr", min_value=1e-5, max_value=1e-2, log_scale=True),
        DiscreteParameterRange("General/batch_size", values=[32, 64, 128]),
        UniformIntegerParameterRange("General/layers", min_value=2, max_value=8),
```



```
objective_metric_sign="max",
optimizer_class=OptimizerOptuna,
execution_queue="gpu",
max_number_of_concurrent_tasks=4,
total_max_jobs=40,
max_iteration_per_job=30,
min_iteration_per_job=5,
time_limit_per_job=45.0,          # minutes
)

optimizer.set_report_period(2.0)    # minutes between progress reports
optimizer.start()                  # or start_locally() to debug
optimizer.wait()
print(optimizer.get_top_experiments(top_k=5))
optimizer.stop()
```

Strategy	Class	When
Random	<code>RandomSearch</code>	A baseline, and a sanity check on your ranges
Grid	<code>GridSearch</code>	Few discrete parameters, exhaustive coverage required
Optuna (TPE)	<code>OptimizerOptuna</code>	The default choice; good with continuous ranges
BOHB	<code>OptimizerBOHB</code>	Expensive training where early stopping saves the most

The parameters that decide whether this saves GPU hours or wastes them:

Setting	Effect
<code>max_number_of_concurrent_tasks</code>	Parallelism. Must fit your worker count, or trials just queue
<code>total_max_jobs</code>	The budget. Set it deliberately, not optimistically
<code>min_iteration_per_job</code>	Trials are protected from pruning until here: too low and you kill slow starters
<code>max_iteration_per_job</code>	Hard cap; the main lever on cost



<code>time_limit_per_job</code>	wait-clock safety net for a run that hangs
<code>objective_metric_*</code>	Must match a reported <code>title</code> / <code>series</code> exactly , or nothing is optimised

The objective metric must match a reported scalar exactly

`objective_metric_title="accuracy"` with `series="val"` only works if your base task calls `report_scalar("accuracy", "val", ...)`. A mismatch does not raise. The optimiser sees no objective, cannot rank anything, and degenerates into random search that never prunes. Run the base task first, look at the Scalars tab, and copy the exact strings.

The base task must be a completed, working run

HPO clones it. If the base task fails, all forty trials fail identically and you have burned a queue. Always validate the base task end to end (including on the agent, not just locally) before pointing an optimiser at it.

► TRY IT

1. Run a base task that reports `accuracy/val` per epoch, and confirm the exact title and series in the UI.
2. Launch an Optuna optimiser with a small budget (`total_max_jobs=8`) against your local agent.
3. Open the optimiser task's Plots tab and read the parallel-coordinates and objective-over-time plots.
4. Now misspell the objective series and rerun. Nothing errors and no pruning happens.

Expected result – a live pruning plot showing weak trials dying early, and, in step four, an optimiser that silently does nothing useful. That silent failure is the single most expensive HPO mistake, because you only notice after the budget is spent.



Beginner created a version and consumed it. The machinery matters now, because dataset decisions are where storage bills and job startup times are decided.

A dataset is a task whose artifacts are **chunked zip archives** plus a file-list state. A child version stores only its own chunks and points at its parents:

a parent chain

Copy

```
iris 1.0.0 — chunk_000.zip (10k rows)
└─ 1.1.0 — chunk_000.zip (+2k rows)      parents: [1.0.0]
    └─ 1.2.0 — chunk_000.zip (relabelled 500) parents: [1.1.0]
```

`get_local_copy()` walks that chain, downloads every needed chunk, and materialises a flattened view. That is cheap for three versions and expensive for three hundred, which is what `squash` is for:

PYTHON

Copy

```
from clearml import Dataset

# Flatten a long chain into one self-contained version
Dataset.squash(
    dataset_name="iris-flat",
    dataset_project="datasets",
    dataset_ids=["id-1.0.0", "id-1.1.0", "id-1.2.0"],
)
```

The operations that matter beyond create/add/finalize:

PYTHON

Copy

```
ds = Dataset.create(dataset_project="datasets", dataset_name="images",
                    parent_datasets=[parent_id])

ds.add_files("./new", wildcard="*.jpg", recursive=True)
ds.remove_files("old/*.jpg") # tombstoned in this child
```



```
ds.get_logger().report_table("class balance", "train", 0, table_plot=balance_df)
ds.set_metadata({"rows": 12000, "labelled_by": "vendor-a"})
print(ds.list_files(), ds.file_count, ds.total_size)
```

Method	Use for
<code>add_files</code>	Copy files into the dataset (uploaded as chunks)
<code>add_external_files</code>	Register files that stay in your bucket: no copy, no duplication
<code>remove_files</code>	Delete in a child version; parents are untouched
<code>get_local_copy</code>	Read-only, cached, shared across tasks on the machine
<code>get_mutable_local_copy</code>	A fresh writable extraction
<code>squash</code>	Collapse a long parent chain into one version
<code>set_metadata</code> / <code>get_logger</code>	Statistics and plots attached to the data itself

`add_external_files` is the right answer for large data

It records paths and hashes without copying bytes, so a 5 TB bucket becomes a versioned dataset without a second 5 TB copy. The trade is that immutability now depends on **your bucket**. If someone overwrites the underlying object, the "immutable" version silently changes. Pair it with bucket versioning and a write-restricted prefix, which Senior covers.

Two operational habits that matter here:

Chunk size is a real tuning knob. Default chunking produces many medium zips. Many small files benefit from larger chunks (fewer requests); a few huge files benefit from smaller ones (better parallelism on download).

Cache placement decides your job startup time. On ephemeral runners the SDK cache is empty every time, so every job re-downloads the dataset. Mount a persistent volume at



► TRY IT

1. Build a five-deep parent chain, then time `get_local_copy()` . Squash it and time the same call again.
2. Register a folder with `add_external_files` and compare the dataset's stored size against the same folder added with `add_files` .
3. Attach a class-balance table to the dataset with `get_logger().report_table` and view it in the UI.
4. Delete your SDK cache directory, fetch a dataset, then fetch again. Compare the two times.

Expected result – squashing turns a chain walk into one download, external files cost almost no storage, and a warm cache turns a multi-minute fetch into nothing. Step three is underused: a dataset that carries its own statistics answers "is this data any good?" without loading it.

Pipelines properly

Once pipelines become the thing you ship, Beginner's three-component example stops being enough.

The controller API, with everything you need for production:

pipeline.py

Copy

```
from clearml import PipelineController

def pre_execute(pipeline, node, params):
    """Runs before a step is enqueued. Return False to skip it."""
    if node.name == "evaluate" and params.get("General/skip_eval"):
        return False
    return True

def post_execute(pipeline, node):
    """Runs after a step finishes – good place for a promotion decision."""
    acc = node.job.task.get_last_scalar_metrics() \
```



```
pipe = PipelineController(  
    name="nightly-train",  
    project="vision",  
    version="1.2.0",  
    add_pipeline_tags=True,          # tag every step with the pipeline id  
    docker="python:3.11-slim",  
    repo="https://github.com/org/repo",  
)  
  
pipe.add_parameter("lr", 3e-4)  
pipe.add_parameter("dataset_version", "1.1.0")  
  
pipe.add_step(  
    name="prepare",  
    base_task_project="vision",  
    base_task_name="make dataset",  
    parameter_override={"General/version": "${pipeline.dataset_version}"},  
    cache_executed_step=True,  
    execution_queue="cpu",  
)  
pipe.add_step(  
    name="train",  
    parents=["prepare"],  
    base_task_project="vision",  
    base_task_name="resnet18 baseline",  
    parameter_override={  
        "General/lr": "${pipeline.lr}",  
        "General/data_id": "${prepare.artifacts.dataset_id.url}",  
    },  
    execution_queue="gpu",  
    retry_on_failure=2,  
    pre_execute_callback=pre_execute,  
    post_execute_callback=post_execute,  
    monitor_metrics=[("accuracy", "val")],  
    monitor_models=["resnet18-cls"],  
)  
pipe.add_step(  
    name="evaluate",  
    parents=["train"],  
    base_task_project="vision",  
    base_task_name="evaluate",  
    parameter_override={"General/model_url": "${train.models.output.-1.url}"},  
    execution_queue="cpu",  
)
```



Feature	What it buys you
<code>cache_executed_step=True</code>	An unchanged step with unchanged inputs is reused, not rerun
<code>retry_on_failure=N</code>	Transient failures (spot preemption, a flaky download) self-heal
<code>pre_execute_callback</code>	Conditional steps: return <code>False</code> to skip
<code>post_execute_callback</code>	Promotion gates and aggregate reporting
<code>monitor_metrics</code>	The step's metric is mirrored onto the pipeline task, so one page shows everything
<code>monitor_models /</code> <code>monitor_artifacts</code>	The pipeline surfaces its steps' outputs directly
<code>add_pipeline_tags=True</code>	Every step is tagged with the pipeline, so a run is filterable as a unit

Caching deserves a precise statement, because people expect it to be smarter than it is: a step is reused when its **base task, its parameters, and its code** are unchanged. Change a parameter and it reruns. Change nothing and it is skipped, and the `previous` run's artifacts are wired into the downstream step. It does not hash your input files, so a step reading a mutable path will reuse a stale result, which is a good argument for versioned datasets rather than paths.

A pipeline controller is a task, so it needs a worker of its own

`pipe.start(queue="services")` enqueues the controller, which then enqueues its steps. If nothing is watching `services`, the pipeline never starts and the UI shows a queued controller with no steps, which reads like a broken pipeline but is a missing worker. `start_locally()` runs the controller in your process instead, which is how you should develop.

Scheduling turns a pipeline into a product:



```
from clearml.automation import TaskScheduler

scheduler = TaskScheduler(sync_frequency_minutes=5)
scheduler.add_task(
    schedule_task_id="pipeline-controller-task-id",
    queue="services",
    minute=0, hour=2,                # 02:00 daily
)
scheduler.start()                  # itself a task, on the services queue
```

There is also a **trigger** scheduler, which is the more interesting one: it fires when a dataset or model changes rather than on a clock.

PYTHON

Copy

```
from clearml.automation import TriggerScheduler

trigger = TriggerScheduler()
trigger.add_dataset_trigger(
    schedule_task_id="pipeline-controller-task-id",
    queue="services",
    trigger_project="datasets",
    trigger_name="iris",
)
trigger.add_model_trigger(
    schedule_task_id="eval-task-id",
    queue="services",
    trigger_project="vision",
    trigger_on_tags=["candidate"],
)
trigger.start()
```

► TRY IT

1. Build a three-step controller pipeline and run it with `start_locally()` . Then run it on the services queue.
2. Rerun it unchanged and confirm the cached step is skipped in the DAG view.
3. Add `monitor_metrics` to the training step and check that the metric appears on the



4. Add a `pre_execute_callback` that skips the evaluation step when a pipeline parameter is set, and prove it works both ways.
5. Register a dataset trigger and then finalize a new dataset version. Watch the pipeline start on its own.

Expected result – step five is the moment a pipeline stops being a script you run and becomes a system that reacts. Step three is the underrated one: a pipeline page that shows its steps' metrics is the difference between one link to share and five.

The model registry as a promotion workflow

Beginner registered a model and tagged it. At this level the registry is a **workflow**, and the tag is the contract between training and serving.

promote.py

Copy

```
from clearml import Model, Task

candidates = Model.query_models(
    project_name="vision",
    model_name="resnet18-cls",
    tags=["candidate"],
    only_published=False,
    max_results=20,
)

def score(model):
    return model.get_metadata().get("val_accuracy", 0)

best = max(candidates, key=score)
current = Model.query_models(project_name="vision", model_name="resnet18-cls",
                             tags=["production"], max_results=1)

if not current or score(best) > score(current[0]) + 0.002:
    if current:
```



```
print("promoted", best.id)
else:
    print("no promotion: improvement below threshold")
```

1 Train

The task saves weights; a model appears, tagged `candidate`, linked to the task and its commit.

2 Evaluate

A separate task loads the candidate by tag and writes metrics into the model's metadata.

3 Gate

A promotion task compares against the current production model with an explicit margin.

4 Promote

Tags move; the winner is published, which makes it read-only.

5 Serve

Serving queries by tag, so deployment is a tag move rather than a code change.

Metadata is the part people skip, and it is what makes the gate writable at all:

PYTHON

Copy

```
from clearml import OutputModel

model = OutputModel(task=task, name="resnet18-cls", framework="PyTorch")
model.update_weights(weights_filename="model.pt", auto_delete_file=False)
model.update_design(config_dict={"arch": "resnet18", "classes": 10, "input": [3, 224, 224]})
model.set_metadata("val_accuracy", 0.941, v_type="float")
model.set_metadata("dataset_version", "1.1.0")
model.set_metadata("commit", task.data.script.version_num)
model.tags = ["candidate"]
```



<code>update_design</code>	The input/output contract, so a consumer can validate shapes
<code>set_metadata</code>	Queryable numbers. This is what a promotion gate reads
<code>tags</code>	The routing mechanism; keep the vocabulary small and enforced
<code>publish()</code>	Immutability. A published model cannot be silently swapped
Task link	Lineage: weights → task → commit → dataset version

Tag vocabulary has to be agreed, or the registry is decoration

If one person writes `prod` , another `production` , and a third `live` , then serving code querying `production` silently finds nothing and keeps the old model. Fix a small vocabulary (`candidate` , `staging` , `production` , `archived`) and set tags from code in the promotion task, never by hand.

► TRY IT

1. Register a model with `set_metadata("val_accuracy", ...)` and confirm it is visible and queryable in the UI.
2. Write the promotion script above and run it against two candidates. Confirm the tags moved.
3. Try to change a published model's weights and read the refusal.
4. Write a loader that fetches `tags=["production"]` and prints the model id, then re-run the promotion and the loader again.

Expected result – a deployment that changed without a code change or a redeploy. Step three matters because "published" is the only thing standing between your gate and someone replacing production weights.

ClearML Serving



BASH

Copy

```
pip install clearml-serving

# 1. A serving service – itself a ClearML task on the services queue
clearml-serving create --name "vision-inference"
# → prints a service id; export it for the compose stack

# 2. Register an endpoint backed by a registry model
clearml-serving --id <service-id> model add \
  --engine triton \
  --endpoint "resnet18" \
  --model-id <model-id> \
  --preprocess "preprocess.py" \
  --input-size 1 3 224 224 --input-type float32 --input-name "input" \
  --output-size 1 10 --output-type float32 --output-name "output"

# 3. Or track a tag instead of an id – new production model, no redeploy
clearml-serving --id <service-id> model auto-update \
  --engine triton --endpoint "resnet18" \
  --project vision --name "resnet18-cls" --tags production --max-versions 2

# 4. Canary between two versions
clearml-serving --id <service-id> model canary \
  --endpoint "resnet18" --weights 0.9 0.1 --input-endpoints resnet18/1 resnet18/2
```

preprocess.py

Copy

```
class Preprocess:
    def preprocess(self, body, state, collect_custom_statistics_fn=None):
        import numpy as np
        arr = np.array(body["image"], dtype=np.float32) / 255.0
        return arr.reshape(1, 3, 224, 224)

    def postprocess(self, data, state, collect_custom_statistics_fn=None):
        probs = data[0].tolist()
        if collect_custom_statistics_fn:
            collect_custom_statistics_fn({"max_prob": max(probs)})
        return {"class": int(max(range(len(probs)), key=probs.__getitem__)), "probs": pr
```



Serving service task	The control plane. Holds the endpoint configuration, lives on services
Inference container	The request path. Reads config, runs preprocess/postprocess
Triton container	GPU inference for framework models that support it
Statistics container	Ships request and drift metrics to Prometheus
auto-update	Endpoint follows a tag, so promotion deploys itself
canary	Weighted split between versions

An inference endpoint is a network service: authenticate it

A ClearML Serving endpoint does not authenticate callers by itself. Put it behind an ingress or API gateway that terminates TLS and enforces authentication, and never expose it directly to the internet. This is the most common oversight when someone stands one up for a demo and then leaves it running.

► TRY IT

1. Serve a small scikit-learn model with the sklearn engine and call it with **curl** .
2. Write a **Preprocess** class that reshapes input and confirm a malformed request fails cleanly rather than with a 500.
3. Switch the endpoint to **auto-update** on a tag, then promote a new model and watch the endpoint pick it up.
4. Enable the statistics container and look at request metrics.

Expected result – step three is the payoff for everything in the previous section: a promotion in the registry becomes a deployment with no pipeline, no redeploy, and no human. Also note in step two how much of production reliability lives in the preprocess code rather than the model.



A task that fails only on the agent is the most annoying failure mode there is. `clearml-session` gives you an interactive shell inside the agent's own environment.

BASH

Copy

```
# A JupyterLab + VS Code server session on a GPU worker
clearml-session --queue gpu \
  --docker nvidia/cuda:12.1.0-runtime-ubuntu22.04 \
  --packages "torch==2.3.0 torchvision" \
  --git-credentials

# Attach to the environment of a specific failed task
clearml-session --queue gpu --base-task-id 8f2c4b19e0a7d3f1c6b8a2e4d7091f3b

# Reconnect to a session you left running
clearml-session --shutdown # when you are done - it holds a worker
```

It prints a local URL tunnelled to the remote machine, with Jupyter, a browser VS Code, and SSH. The environment is the `same` one the agent builds for a task, which is the whole point: you are debugging the actual failure, not an approximation of it.

Flag	Use
<code>--base-task-id</code>	Reproduce a specific task's environment exactly
<code>--docker</code>	Pick the image, and therefore CUDA
<code>--packages</code>	Extras beyond the base task's list
<code>--git-credentials</code>	Forward your git credentials for private repos
<code>--queue</code>	Which hardware you want to sit on
<code>--shutdown</code>	Release the worker. Sessions hold a GPU until stopped

A session occupies a worker until you shut it down



This is the number one source of why is the GPU queue full of nothing in teams that have just discovered the feature.

► TRY IT

1. Start a session on your local agent and open the Jupyter URL it prints.
2. Inside the session, run `pip list` and compare it against a task's Installed Packages.
3. Take a task that failed on the agent and start a session with `--base-task-id`. Reproduce the failure interactively.
4. Shut it down and confirm the worker returns to idle in Workers & Queues.

Expected result – the failure reproduced in the environment that produced it, in a browser, in under two minutes. Step three converts the worst class of ClearML bug, "only breaks on the agent", into an ordinary debugging session.

ClearML in CI

The pattern worth building: a pull request that changes training code launches a short training run, and the check passes only if the metric clears a threshold. That makes model quality a reviewable property of the diff.

.github/workflows/train.yml

Copy

```
name: train
on:
  pull_request:
    paths: ['src/**', 'configs/**', 'requirements.txt']

permissions:
  contents: read

jobs:
  smoke-train:
```



```
with:
  fetch-depth: 0          # the agent needs a real commit to clone

- uses: actions/setup-python@v5
  with:
    python-version: '3.11'
    cache: pip

- run: pip install -r requirements.txt

- name: Launch a tracked run on the GPU queue
  env:
    CLEARML_API_HOST: ${ secrets.CLEARML_API_HOST }
    CLEARML_WEB_HOST: ${ secrets.CLEARML_WEB_HOST }
    CLEARML_FILES_HOST: ${ secrets.CLEARML_FILES_HOST }
    CLEARML_API_ACCESS_KEY: ${ secrets.CLEARML_API_ACCESS_KEY }
    CLEARML_API_SECRET_KEY: ${ secrets.CLEARML_API_SECRET_KEY }
  run: python ci/launch_and_gate.py --epochs 3 --min-accuracy 0.85
```

ci/launch_and_gate.py

Copy

```
import argparse, os, sys, time
from clearml import Task

ap = argparse.ArgumentParser()
ap.add_argument("--epochs", type=int, default=3)
ap.add_argument("--min-accuracy", type=float, default=0.85)
args = ap.parse_args()

task = Task.create(
    project_name="vision/ci",
    task_name=f"pr smoke {os.environ.get('GITHUB_SHA', 'local')[:8]}",
    script="src/train.py",
    repo=os.environ["GITHUB_SERVER_URL"] + "/" + os.environ["GITHUB_REPOSITORY"],
    commit=os.environ["GITHUB_SHA"],
    requirements_file="requirements.txt",
    docker="nvidia/cuda:12.1.0-runtime-ubuntu22.04",
)

task.set_parameters({"General/epochs": args.epochs})
task.add_tags(["ci", f"pr-{os.environ.get('GITHUB_REF_NAME', 'local')}"])
Task.enqueue(task, queue_name="gpu")
```



```
if task.status != "completed":
    sys.exit(f"training task {task.status}: {task.get_output_log_web_page()}")

acc = task.get_last_scalar_metrics().get("accuracy", {}).get("val", {}).get("last", 0)
print(f"::notice::val accuracy {acc:.4f} - {task.get_output_log_web_page()}")
if acc < args.min_accuracy:
    sys.exit(f"accuracy {acc:.4f} below threshold {args.min_accuracy}")
```

CI decision	Why
<code>fetch-depth: 0</code>	The agent clones the commit; a shallow checkout can leave it unfetchable
<code>Task.create</code> + <code>enqueue</code> , not <code>Task.init</code>	The runner should not do the training; it launches and waits
Credentials as secrets, in env vars	There is no <code>clearml.conf</code> to leak, and rotation is one place
Short epoch budget on pull requests	The gate is "did this break", not "is this the best model"
The web page URL printed into the log	A reviewer gets one click to the full run
A tag per pull request	Every CI run is filterable, and cleanable, as a group

Fork pull requests must not get a write credential

A ClearML access/secret pair can enqueue tasks that execute arbitrary code on your agents. On a public repository, `pull_request` from a fork with access to that secret is remote code execution on your GPU fleet. Use `pull_request_target` with an explicit review gate, or restrict fork builds to a queue whose agents are ephemeral and hold no other credentials. Senior treats this properly.

► TRY IT

1. Add the launch-and-gate script to a repository and run it locally with your own credentials first.



4. Check what happens if the queue has no agent: how long does your CI job wait before it times out?

Expected result – a red check with a link to a full experiment page. Step four is the operational detail people miss: without a timeout, a missing agent turns into a CI job that burns runner minutes for six hours.

Reports and sharing

A tracked experiment is only useful if someone reads it. ClearML Reports are markdown documents that embed **live** task resources, so a plot in a report updates when the task does.

Element	Use
Embedded scalar plot	The metric that justifies the decision, live
Embedded task	The full run, one click away, with all its config
Embedded model	The exact artifact under discussion
Markdown prose	The reasoning, which is the part nobody writes down

The pieces you need in code to make a report worth writing:

PYTHON

Copy

```
# A plot the report can embed
logger.report_table("per-class F1", "test", 0, table_plot=f1_df)

# A one-line summary a reader sees without opening anything
logger.report_text(
    f"Baseline 0.921 → 0.940 on iris 1.1.0. "
    f"Change: depth 4 → 8. Cost: +6 min/epoch."
)
```



Write the number into the task name or a text report

"resnet18 depth=8 acc=0.940" costs nothing and makes the experiments table self-documenting. A month later, a table where every row states its own result is the difference between reading a list and doing archaeology.

► TRY IT

1. Create a report, embed a scalar plot from a running task, and watch the embed update.
2. Embed a comparison of three tasks and share the link with someone who has never opened ClearML.
3. Add a `report_text` summary to your training script and confirm it shows up in the task.

Expected result – a document that stays correct as the experiment continues. The person in step two is the real test: if they cannot understand the result from your report alone, the report is missing the reasoning rather than the plots.

Debugging, one level deeper

Beginner had a tab order. When the tabs run out, work through these.

01 Turn up SDK logging

`CLEARML_LOG_LEVEL=DEBUG` shows every API call the SDK makes, which is how you diagnose a hang: you can see which request never returned.

02 Read the agent's own log, not the task's

Setup failures (pip resolution, docker pull, git fetch) happen before your code runs and appear in the agent's stdout, not in the task console.



04 Compare a working and a broken task in the UI

Details view with "hide identical values" narrows a mystery to a package version or a single parameter in seconds.

05 Check the diff, not just the commit

Two tasks on the same commit can differ by the stored uncommitted patch.

`task.data.script.diff` shows it.

06 Verify the server is not the problem

`curl $CLEARML_API_HOST/debug.ping` . Elasticsearch under pressure looks like the SDK "hanging" on metric reports.

BASH

Copy

```
export CLEARML_LOG_LEVEL=DEBUG          # verbose SDK
export CLEARML_AGENT_LOG_LEVEL=DEBUG     # verbose agent
clearml-agent daemon --queue gpu --foreground # do not detach; watch it live
curl -s "$CLEARML_API_HOST/debug.ping"
```

compare two tasks programmatically

Copy

```
from clearml import Task

good = Task.get_task(task_id="...good...")
bad  = Task.get_task(task_id="...bad...")

gp, bp = good.get_parameters(), bad.get_parameters()
for key in sorted(set(gp) | set(bp)):
    if gp.get(key) != bp.get(key):
        print(f"{key}: {gp.get(key)!r} → {bp.get(key)!r}")

print("commit:", good.data.script.version_num, "→", bad.data.script.version_num)
```



Setup fails, task console empty	Failure happened before your code ran	Read the agent's log
Task hangs at start with no output	API host unreachable, or a proxy	<code>CLEARML_LOG_LEVEL=DEBUG ; debug.ping</code>
Scalars lag minutes behind	Elasticsearch under load, or a huge report rate	Report less often; check the server
Two runs on one commit behave differently	Different stored diffs	Compare <code>data.script.diff</code>
Docker pull fails on the agent	No registry credentials on that host	<code>docker login</code> as the agent's user
Works in session, fails as a task	The entry point or working directory differs	Check Execution ▶ script and cwd

► TRY IT

1. Run a script with `CLEARML_LOG_LEVEL=DEBUG` and read the API calls `Task.init` makes.
2. Deliberately break the docker image name on a task and find where the error appears: task console or agent log?
3. Use the programmatic diff above on two of your own tasks that should be identical.
4. Point `CLEARML_API_HOST` at a wrong port and observe how the hang presents.

Expected result – step two is the important habit: setup failures are invisible in the UI, and knowing to read the agent's log first saves the half hour you would otherwise spend refreshing the console tab.

Putting it all together

A complete project using everything on this page. Nothing here is new. Read it as a whole and you should be able to justify every line.



```
.
├── .github/workflows/train.yml      # PR gate: launch, wait, threshold
├── requirements.txt                 # pinned, and the agent's source of truth
├── configs/
│   ├── model.yaml                  # connect_configuration
│   └── features.yaml
├── src/
│   ├── make_dataset.py             # data_processing task → Dataset version
│   ├── train.py                    # training task, execute_remotely
│   └── evaluate.py                 # qc task, writes model metadata
├── ops/
│   ├── pipeline.py                 # PipelineController, services queue
│   ├── hpo.py                      # HyperParameterOptimizer
│   ├── promote.py                  # tag-moving gate
│   └── triggers.py                 # TriggerScheduler on the dataset
└── ci/launch_and_gate.py
```

src/train.py

Copy

```
import os
from clearml import Task

task = Task.init(
    project_name="vision/experiments",
    task_name="resnet18",
    task_type=Task.TaskTypes.training,
    output_uri="s3://ml-artifacts/clearml",
    reuse_last_task_id=False,
)
task.add_tags(["resnet18"])
task.set_packages("requirements.txt") # explicit, not detected
task.set_base_docker("nvidia/cuda:12.1.0-runtime-ubuntu22.04",
    docker_arguments="--shm-size=8g")

import torch
from clearml import Dataset, OutputModel

opt = task.connect({"lr": 3e-4, "epochs": 20, "seed": 42}, name="optimizer")
arch = task.connect({"layers": 18, "width": 64}, name="architecture")
data = task.connect({"dataset_version": "1.1.0"}, name="data")
model_cfg_path = task.connect_configuration("configs/model.yaml", name="model")
```



```
root = Dataset.get(
    dataset_project="datasets", dataset_name="iris",
    dataset_version=data["dataset_version"], alias="training data",
).get_local_copy()

logger = task.get_logger()
for epoch in range(opt["epochs"]):
    tr, va, acc = train_one_epoch(root, opt, arch)
    logger.report_scalar("loss", "train", tr, epoch)
    logger.report_scalar("loss", "val", va, epoch)
    logger.report_scalar("accuracy", "val", acc, epoch) # the HPO objective

test_acc = test(root)
logger.report_single_value("test_accuracy", test_acc)
task.upload_artifact("predictions", predictions_df)

model = OutputModel(task=task, name="resnet18-cls", framework="PyTorch")
model.update_weights(weights_filename="model.pt")
model.update_design(config_dict={"arch": "resnet18", "input": [3, 224, 224]})
model.set_metadata("val_accuracy", test_acc, v_type="float")
model.set_metadata("dataset_version", data["dataset_version"])
model.tags = ["candidate"]
```

ops/pipeline.py

Copy

```
from clearml import PipelineController

pipe = PipelineController(name="nightly", project="vision", version="1.2.0",
    add_pipeline_tags=True, docker="python:3.11-slim")
pipe.add_parameter("lr", 3e-4)
pipe.add_parameter("dataset_version", "1.1.0")

pipe.add_step(name="prepare", execution_queue="cpu", cache_executed_step=True,
    base_task_project="vision", base_task_name="make dataset",
    parameter_override={"General/version": "${pipeline.dataset_version}"})

pipe.add_step(name="train", parents=["prepare"], execution_queue="gpu",
    base_task_project="vision/experiments", base_task_name="resnet18",
    parameter_override={"optimizer/lr": "${pipeline.lr}",
        "data/dataset_version": "${pipeline.dataset_version}"},
    retry_on_failure=2,
    monitor_metrics=[("accuracy", "val")],
```



```
base_task_project="vision", base_task_name="evaluate",
parameter_override={"General/model_url": "${train.models.output.-1.url}"}

pipe.add_step(name="promote", parents=["evaluate"], execution_queue="services",
base_task_project="vision", base_task_name="promote")

pipe.start(queue="services")
```

BASH

Copy

```
# The fleet
clearml-agent daemon --queue services --services-mode --cpu-only --detached
clearml-agent daemon --queue cpu --cpu-only --detached
clearml-agent daemon --queue gpu --gpus 0 --docker nvidia/cuda:12.1.0-runtime-ubuntu22.0
clearml-agent daemon --queue gpu --gpus 1 --docker nvidia/cuda:12.1.0-runtime-ubuntu22.0

# Search, then ship
python ops/hpo.py # 40 trials, 4 at a time, pruned
python ops/triggers.py # new dataset version → pipeline runs
clearml-serving --id $SVC model auto-update --endpoint resnet18 \
--project vision --name resnet18-cls --tags production --max-versions 2
```

Twelve decisions in there are the whole lesson of this page:

Decision	Section
<code>set_packages</code> instead of trusting detection	Automatic capture: the mechanism
Docker image pinned on the task	Automatic capture: the mechanism
Config split into named sections	Configuration: sections and objects
YAML through <code>connect_configuration</code>	Configuration: sections and objects
<code>reuse_last_task_id=False</code> on every experiment	Task creation and lifecycle
Dataset version as a parameter, with <code>alias</code>	Datasets at scale
The HPO objective is a real reported series	Hyperparameter optimisation



model metadata written for the gate to read	the registry as a promotion workflow
<code>cache_executed_step</code> on the deterministic step	Pipelines properly
<code>retry_on_failure</code> on the expensive step	Pipelines properly
Controllers on <code>services</code> , never on <code>gpu</code>	Agents in depth
Serving follows a tag, so promotion deploys	ClearML Serving

► TRY IT: THE ONE THAT MATTERS

1. Take this layout into a real project. Get the training task green on an agent, in docker mode, with an explicit package list.
2. Run the HPO with a small budget and confirm pruning happens by checking the objective plot, not just the task count.
3. Build the pipeline and run it twice. Confirm the cached step is skipped the second time.
4. Wire the dataset trigger, then finalize a new dataset version and let the whole thing run unattended.
5. Promote by tag and confirm your serving endpoint picks up the new model with no deploy step.

Expected result – an end-to-end system where new data produces a trained, evaluated, promoted, and served model without anyone typing a command. Step four is the acceptance test; if it needs a human anywhere, find that spot and close it.

Where you are now

You can control task creation and reuse precisely, fix requirement detection instead of fighting it, structure configuration so a clone is fully reconfigurable, tune agent modes and caches so setup is seconds rather than minutes, launch code that has no ClearML in it, run a real HPO with a correct objective and a budget, manage datasets at a scale where chunking and squashing matter, build pipelines with caching, retries, callbacks, and triggers, run the registry



Can you...	
Distinguish <code>init</code> , <code>create</code> , and <code>clone</code> ?	Runs now / builds for someone else / editable copy
Say why requirement detection misses a package?	It inspects <code>sys.modules</code> ; a lazy import is never there
Name the agent's four caches?	pip, venv, vcs, and the SDK storage cache
Say why <code>system_site_packages</code> matters?	Trust the image; setup drops from minutes to seconds
Explain <code>--services-mode</code> ?	Many concurrent light tasks; controllers must not hold a GPU
Write a correct HPO objective?	Exact <code>title</code> / <code>series</code> from the base task's Scalars tab
Say what pipeline caching keys on?	Base task, parameters, and code: not input file contents
Name the reference syntax for a step's model?	<code>\${step.models.output.-1.url}</code>
Explain how promotion reaches serving?	<code>auto-update</code> follows a tag; a tag move is a deploy
Say what <code>clearml-session --base-task-id</code> gives you?	A shell in the exact environment that failed
Name the first log to read on a setup failure?	The agent's own log, not the task console
Say why fork PRs must not hold ClearML credentials?	Enqueue is remote code execution on your agents

Senior takes every one of those topics further: the self-hosted deployment and its three data stores, the credential and access model across workspaces, multi-tenancy and quota, storage cost and retention policy, Elasticsearch and MongoDB scaling and what breaks first, backup and upgrade procedure, audit and lineage for regulated work, GPU fleet economics and



PREVIOUS

Beginner · Tips & Tricks

NEXT UP

Mid-level · Interview Review



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

Roadmaps

Student Guides

Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY

Meet the team



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.